EXPERIMENT NO: 1

Install the Flutter SDK on your system and configure the required tools for Flutter development. Install Visual Studio Code (or Android Studio), configure the Flutter and Dart extensions, verify the installation using Flutter Doctor, and create and execute your first Flutter application.

Aim

To install and configure the Flutter SDK and required development tools, verify the installation using Flutter Doctor, and create and execute a Flutter application.

Theory

Flutter is an open-source UI framework developed by Google for building cross-platform applications using the Dart programming language. The Flutter SDK provides the tools required to develop, test, and run Flutter applications. Visual Studio Code can be used as the development environment with Flutter and Dart extensions. The flutter doctor command is used to check whether the required tools and dependencies are properly configured.

Requirements

- Flutter SDK
- Dart SDK
- Visual Studio Code
- Flutter Extension
- Dart Extension
- Chrome/Android Emulator

Algorithm

1. Start.
2. Download and install the Flutter SDK.
3. Add the Flutter SDK bin directory to the system PATH.
4. Install Visual Studio Code.
5. Install the Flutter and Dart extensions in Visual Studio Code.
6. Open the terminal and verify the Flutter installation using flutter --version.
7. Run flutter doctor to check the Flutter development environment.
8. Create a new Flutter application using the flutter create command.
9. Open the created project in Visual Studio Code.
10. Execute the Flutter application using the flutter run command.
11. Stop.

Commands Used

```
flutter --version
flutter doctor
flutter create my_first_app
cd my_first_app
flutter run -d chrome
```

PROCEDURE

1. Install Git

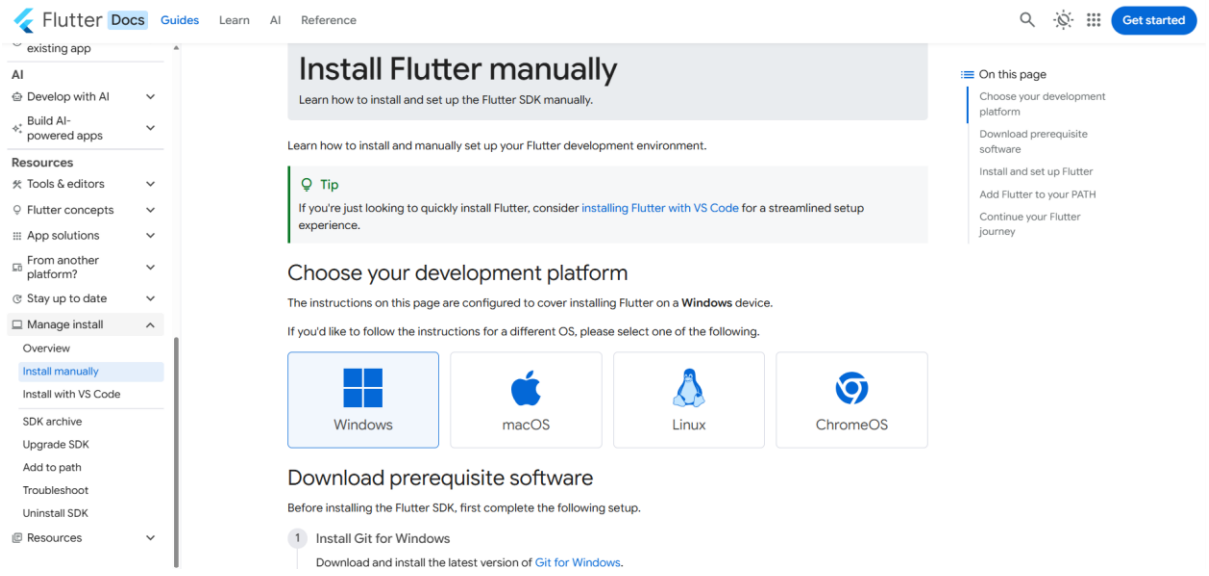
Download and install Git on the system. Git is used by Flutter for development and package management.

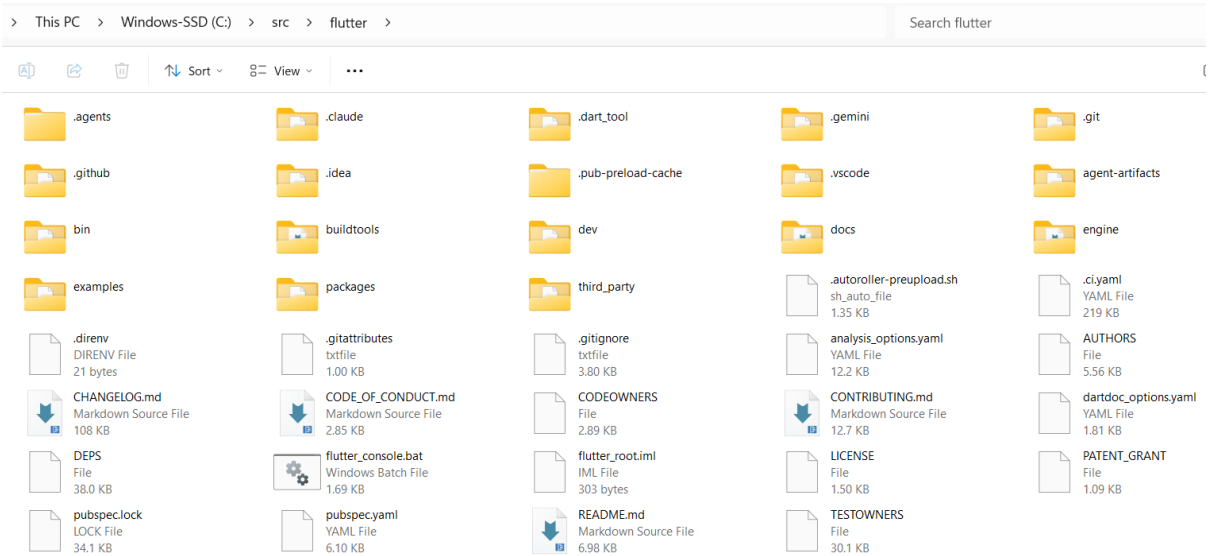
```
C:\Users\LENOVO>git --version
git version 2.51.2.windows.1
```

2. Download Flutter SDK

Download the Flutter SDK for Windows and extract it to a suitable location:

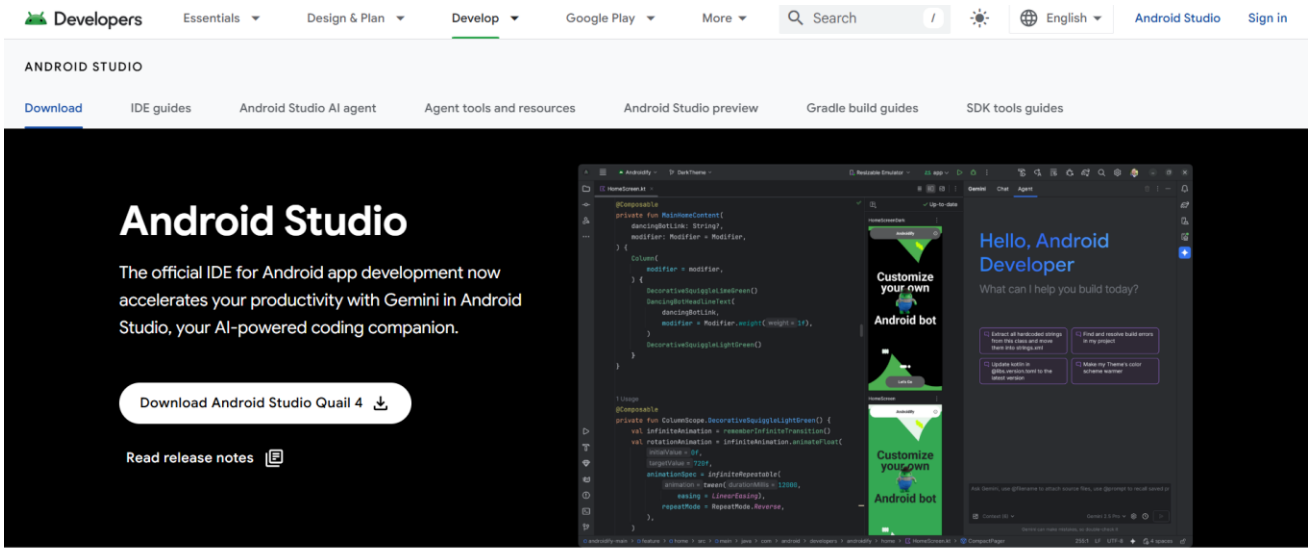
C:\src\flutter

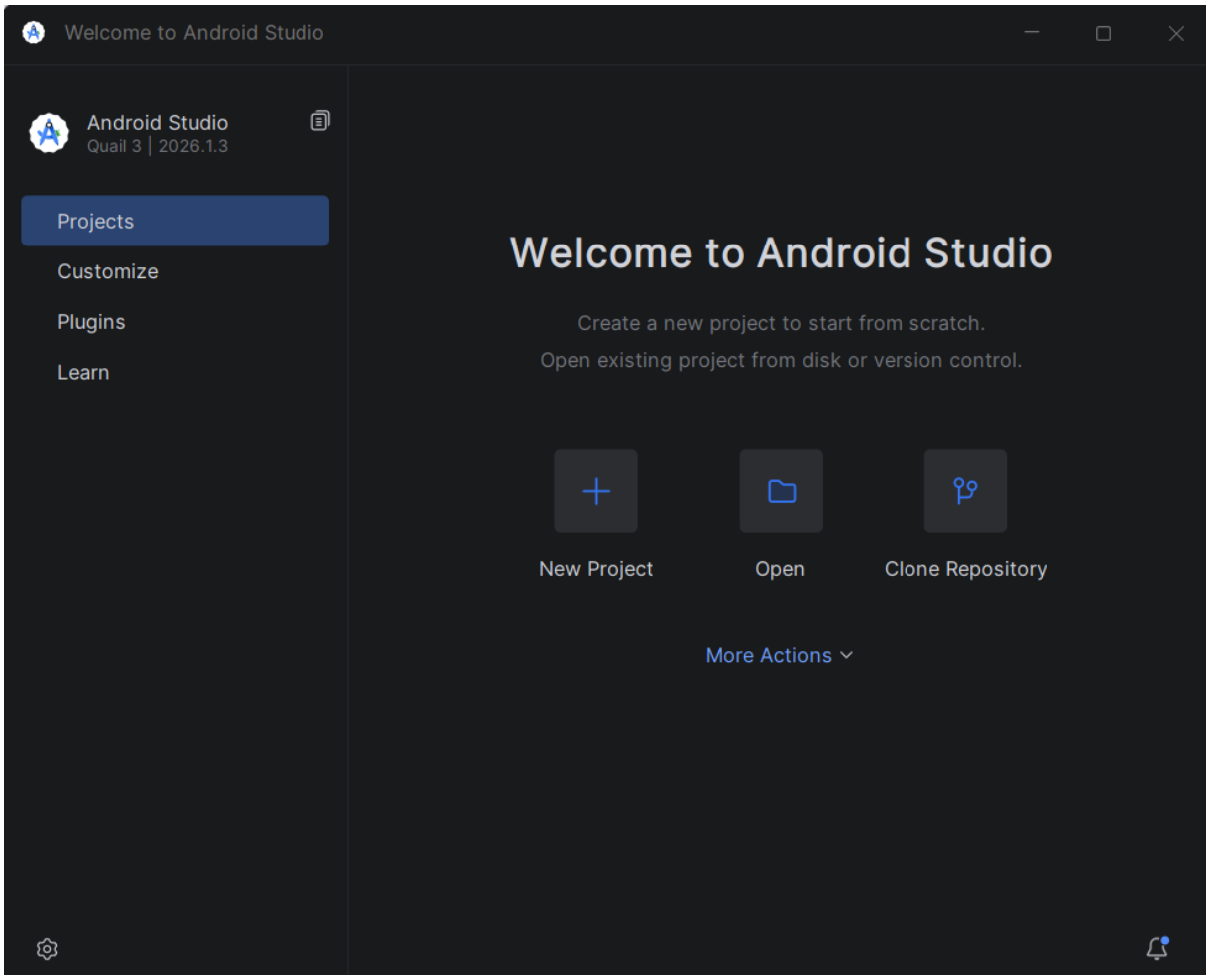




3. Install Android Studio

Download and install Android Studio. Configure the Android SDK, SDK Command-line Tools, SDK Build-Tools, and an Android Virtual Device for Android application development and testing.



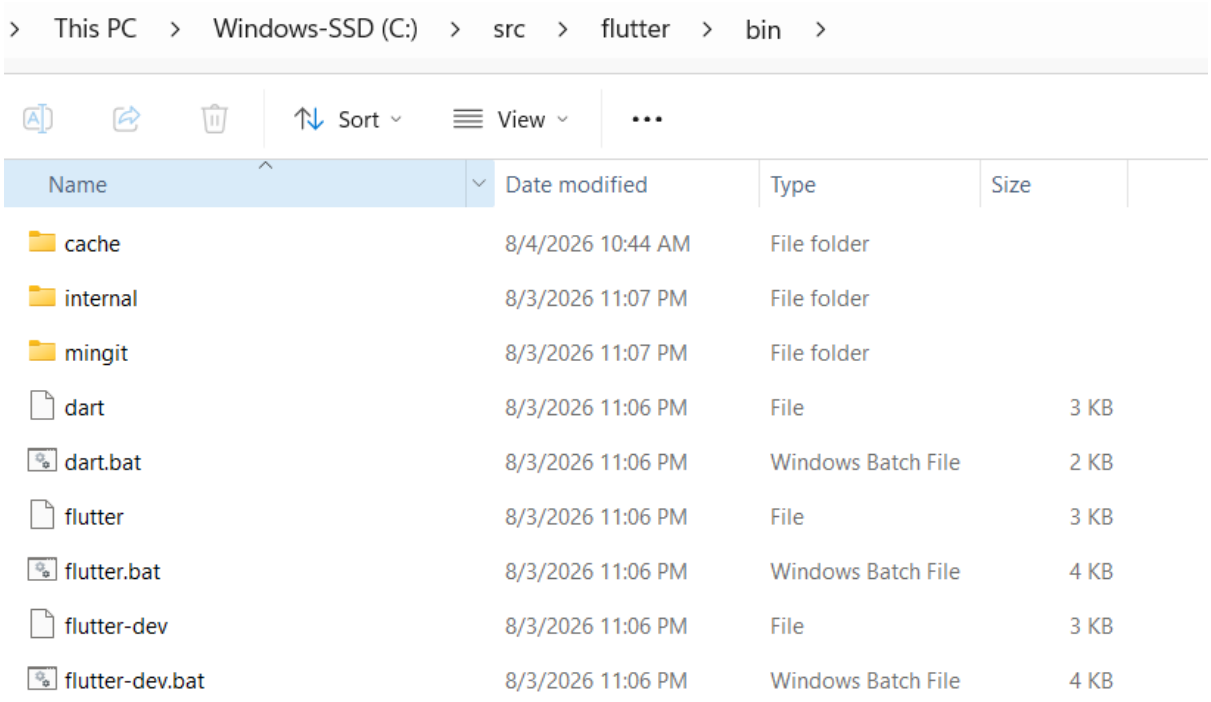


4. Configure Environment PATH

Add the Flutter SDK bin directory to the system PATH:

C:\src\flutter\bin

This allows Flutter commands to be executed from the terminal.



5. Verify Flutter Installation

Open Command Prompt or PowerShell and execute:

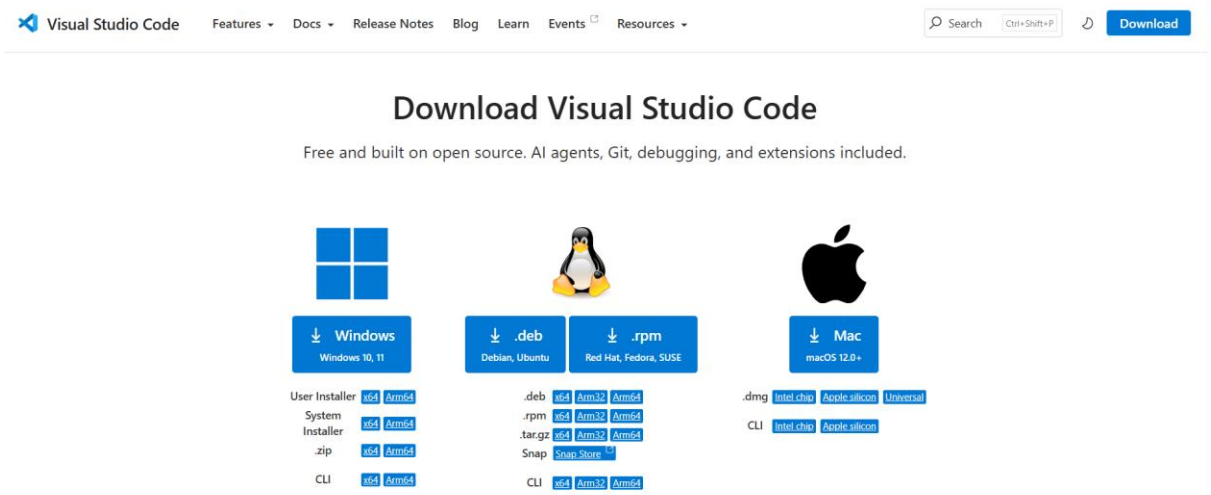
flutter --version

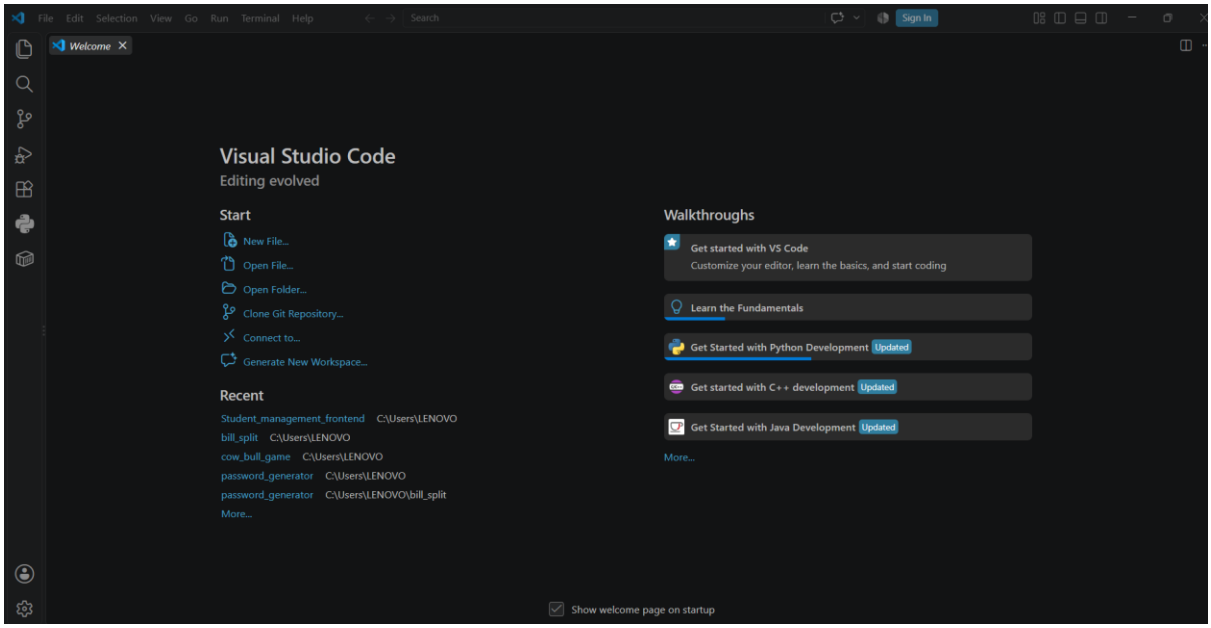
The installed Flutter and Dart versions are displayed.



6. Install Visual Studio Code

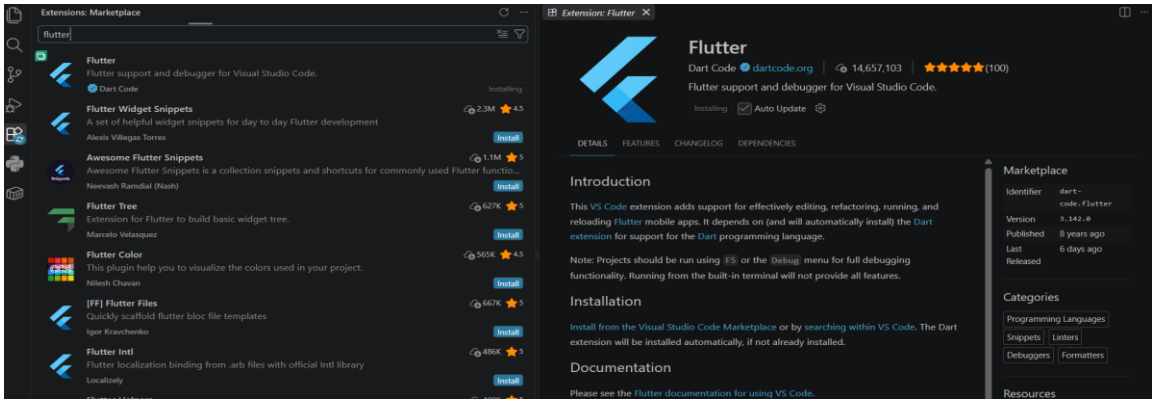
Download and install Visual Studio Code as the development environment for Flutter application development.





7. Install Flutter and Dart Extensions

Open Visual Studio Code and install the **Flutter** extension. The Dart extension is installed automatically as a dependency. Restart Visual Studio Code after installation.



8. Run Flutter Doctor

Execute:

flutter doctor

Flutter Doctor checks the Flutter development environment and reports the status of the available dependencies.

If Android development issues or license issues are reported, execute:

flutter doctor --android-licenses

Accept the required Android SDK licenses.

```
PS C:\Users\LENOVO> flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[√] Flutter (Channel stable, 3.44.8, on Microsoft Windows [Version 10.0.26200.9168], locale en-US)
[√] Windows Version (Windows 11 or higher, 25H2, 2009)
[√] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[√] Chrome - develop for the web
[√] Visual Studio - develop Windows apps (Visual Studio Build Tools 2022 17.14.25 (January 2026))
[√] Connected device (3 available)
[√] Network resources

• No issues found!
```

9. Create the First Flutter Application

Create a new Flutter project:

flutter create my_first_app

cd my_first_app

code .

The Flutter project is created successfully and opened in Visual Studio Code.

```
PS C:\Users\LENOVO> flutter create my_first_app
Creating project my_first_app...
Resolving dependencies in `my_first_app`... (2.2s)
Downloading packages... (5.8s)
Got dependencies in `my_first_app`.
Wrote 131 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your application, type:

  $ cd my_first_app
  $ flutter run

Your application code is in my_first_app\lib\main.dart.
```

```
PS C:\Users\LENOVO> cd my_first_app
PS C:\Users\LENOVO\my_first_app> code .
```

main.dart > HelloWorldApp

```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(const HelloWorldApp());
}

class HelloWorldApp extends StatelessWidget {
  const HelloWorldApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Hello World',
      theme: ThemeData(
        useMaterial3: true,
      ), // ThemeData
      home: const HelloWorldPage(),
    ); // MaterialApp
  }
}

class HelloWorldPage extends StatelessWidget {
  const HelloWorldPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text(
          'My First Flutter App',
          style: TextStyle(
            fontWeight: FontWeight.bold,
          ), // TextStyle
        ), // Text
        centerTitle: true,
```

10. Check Available Devices

Execute:

flutter devices

The available devices such as Chrome, Android emulator, or connected physical devices are displayed.

```
PS C:\Users\LENOVO\my_first_app> flutter devices
Found 3 connected devices:
  Windows (desktop) • windows • windows-x64    • Microsoft Windows [Version 10.0.26200.9168]
  Chrome (web)      • chrome  • web-javascript • Google Chrome 152.0.7977.76
  Edge (web)        • edge    • web-javascript • Microsoft Edge 152.0.4191.66

Run "flutter emulators" to list and start any available device emulators.

If you expected another device to be detected, please run "flutter doctor" to diagnose potential issues. You may also try increasing the
time to wait for connected devices with the "--device-timeout" flag. Visit https://flutter.dev/setup/ for troubleshooting tips.
```

11. Execute the Hello World Application

To run the application on Chrome:

flutter run -d chrome

For Android, run:

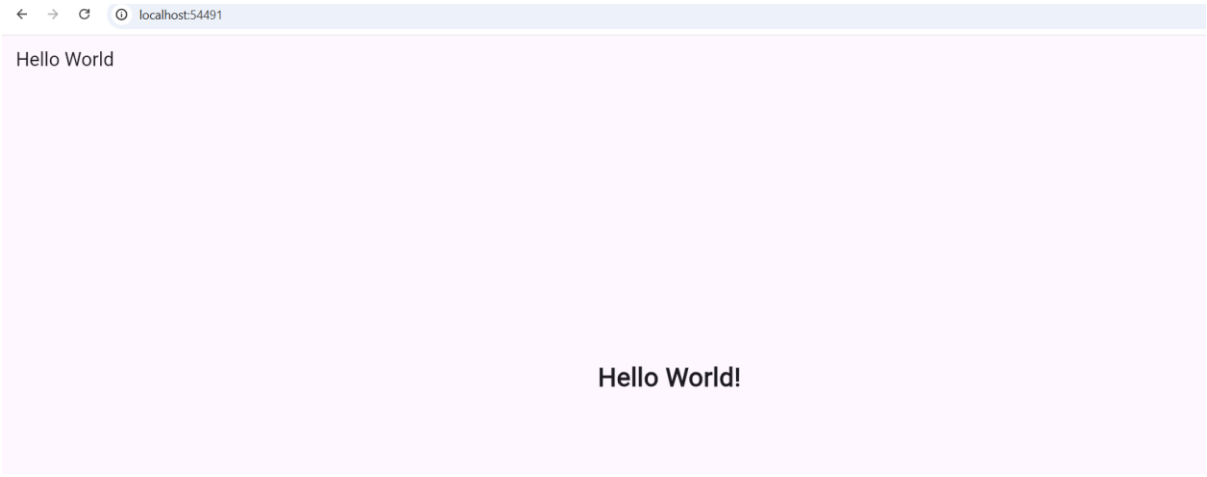
flutter run

and select the available Android emulator or physical device.

```
PS C:\Users\LENOVO\my_first_app> flutter run -d chrome
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...            27.2s

Flutter run key commands.
r Hot reload.
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).

This app is linked to the debug service: ws://127.0.0.1:57392/GxUIkt4QpZU=/ws
Debug service listening on ws://127.0.0.1:57392/GxUIkt4QpZU=/ws
A Dart VM Service on Chrome is available at: http://127.0.0.1:57392/GxUIkt4QpZU=
The Flutter DevTools debugger and profiler on Chrome is available at:
http://127.0.0.1:57392/GxUIkt4QpZU=/devtools/?uri=ws://127.0.0.1:57392/GxUIkt4QpZU=/ws
Starting application from main method in: org-dartlang-app:/web_entrypoint.dart.
```



RESULT

Thus, the Flutter SDK and required development tools were successfully installed and configured. The installation was verified using Flutter Doctor, and a Hello World Flutter application was successfully created and executed.

EXPERIMENT NO: 2

Design a Flutter application to implement a Simple Calculator. The application should accept two numbers, perform Addition, Subtraction, Multiplication, and Division, and display the result. Also include a Clear button to reset the input fields and the result. Use appropriate Flutter widgets to build the user interface.

Aim

To develop a Flutter application that accepts two numbers and performs Addition, Subtraction, Multiplication, and Division, with a Clear button to reset the input fields and result.

Theory

A calculator is a simple application that performs arithmetic operations on user-provided numbers. In Flutter, widgets such as TextField, ElevatedButton, Text, Column, and Scaffold can be used to construct the user interface. TextEditingController is used to read and clear the values entered in the input fields. The setState() method is used to update and display the calculated result dynamically.

Algorithm

1. Start.
2. Create the Flutter application.
3. Create two input fields to accept two numbers.
4. Read the values entered by the user.
5. Convert the input values into numbers.
6. Display buttons for Addition, Subtraction, Multiplication, Division, and Clear.
7. If Addition is selected, add the two numbers and display the result.

8. If Subtraction is selected, subtract the second number from the first and display the result.
9. If Multiplication is selected, multiply the two numbers and display the result.
10. If Division is selected, divide the first number by the second and display the result.
11. If the second number is zero during division, display an appropriate error message.
12. If Clear is selected, reset both input fields and the result.
13. Stop.

Program

```
import 'package:flutter/material.dart';


void main() {
  runApp(const CalculatorApp());
}


class CalculatorApp extends StatelessWidget {
  const CalculatorApp({super.key});


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Simple Calculator',
      theme: ThemeData(
        useMaterial3: true,
        fontFamily: 'Arial',
      ),
      home: const CalculatorPage(),
```

```
    );
  }
}

class CalculatorPage extends StatefulWidget {
  const CalculatorPage({super.key});

  @override
  State<CalculatorPage> createState() => _CalculatorPageState();
}

class _CalculatorPageState extends State<CalculatorPage> {
  // Controllers for the two input fields
  final TextEditingController firstController =
      TextEditingController();

  final TextEditingController secondController =
      TextEditingController();

  // Stores the calculated result
  String result = '0';

  // Performs the selected arithmetic operation
  void calculate(String operation) {
    double? num1 = double.tryParse(firstController.text);
    double? num2 = double.tryParse(secondController.text);

    // Check whether valid numbers are entered
    if (num1 == null || num2 == null) {
      setState(() {
        result = 'Enter valid numbers';
      });
```

```
    return;
  }

  double answer;

  switch (operation) {
    case '+':
      answer = num1 + num2;
      break;

    case '-':
      answer = num1 - num2;
      break;

    case '×':
      answer = num1 * num2;
      break;

    case '÷':
      // Prevent division by zero
      if (num2 == 0) {
        setState(() {
          result = 'Cannot divide by zero';
        });
        return;
      }

      answer = num1 / num2;
      break;

    default:
      return;
```

```
    }

    // Display the calculated result
    setState(() {
      result = formatResult(answer);
    });
  }

  // Formats the result
  String formatResult(double value) {
    // Display whole numbers without decimal places
    if (value == value.toInt()) {
      return value.toInt().toString();
    }

    // Display decimal results up to two decimal places
    return value.toStringAsFixed(2);
  }

  // Clears both input fields and the result
  void clearCalculator() {
    firstController.clear();
    secondController.clear();

    setState(() {
      result = '0';
    });
  }

  @override
  void dispose() {
    firstController.dispose();
```

```
    secondController.dispose();
    super.dispose();
  }

  // Creates a styled operation button
  Widget operationButton(
    String text,
    IconData icon,
    VoidCallback onPressed,
  ) {
    return Expanded(
      child: Padding(
        padding: const EdgeInsets.all(6),
        child: SizedBox(
          height: 70,
          child: ElevatedButton(
            onPressed: onPressed,
            style: ElevatedButton.styleFrom(
              backgroundColor: const Color(0xFF334155),
              foregroundColor: Colors.white,
              elevation: 0,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(18),
              ),
            ),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Icon(
                  icon,
                  size: 24,
                ),
```

```
              const SizedBox(height: 4),
              Text(
                text,
                style: const TextStyle(
                  fontSize: 14,
                  fontWeight: FontWeight.w600,
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  );
}

// Creates a styled input field
Widget numberField(
  String label,
  String hint,
  TextEditingController controller,
  IconData icon,
) {
  return TextField(
    controller: controller,

    keyboardType: const TextInputType.numberWithOptions(
      decimal: true,
      signed: true,
    ),

    style: const TextStyle(
```

```
      color: Colors.white,
      fontSize: 18,
      fontWeight: FontWeight.w500,
    ),

    decoration: InputDecoration(
      labelText: label,
      hintText: hint,

      prefixIcon: Icon(
        icon,
        color: const Color(0xFF38BDF8),
      ),

      labelStyle: const TextStyle(
        color: Color(0xFF94A3B8),
      ),

      hintStyle: const TextStyle(
        color: Color(0xFF64748B),
      ),

      filled: true,
      fillColor: const Color(0xFF1E293B),

      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(18),
        borderSide: BorderSide.none,
      ),

      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(18),
```

```
        borderSide: const BorderSide(
          color: Color(0xFF38BDF8),
          width: 2,
        ),
      ),
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFF0F172A),

    body: SafeArea(
      child: Center(
        child: SingleChildScrollView(
          child: ConstrainedBox(
            constraints: const BoxConstraints(
              maxWidth: 500,
            ),

            child: Padding(
              padding: const EdgeInsets.all(24),

              child: Column(
                children: [

                  // --------------------------------
                  // HEADER
                  // --------------------------------
```

```
const Icon(
  Icons.calculate_rounded,
  color: Color(0xFF38BDF8),
  size: 48,
),

const SizedBox(height: 10),

const Text(
  'Simple Calculator',
  style: TextStyle(
    color: Colors.white,
    fontSize: 30,
    fontWeight: FontWeight.bold,
  ),
),

const SizedBox(height: 6),

const Text(
  'Perform basic arithmetic operations',
  style: TextStyle(
    color: Color(0xFF94A3B8),
    fontSize: 14,
  ),
),

const SizedBox(height: 30),

// --------------------------------
// RESULT DISPLAY
// --------------------------------
```

```
Container(
  width: double.infinity,
  padding: const EdgeInsets.all(24),

  decoration: BoxDecoration(
    gradient: const LinearGradient(
      colors: [
        Color(0xFF1E293B),
        Color(0xFF172554),
      ],
    ),

    borderRadius: BorderRadius.circular(25),
  ),

  child: Column(
    crossAxisAlignment:
        CrossAxisAlignment.end,

    children: [

      const Text(
        'RESULT',
        style: TextStyle(
          color: Color(0xFF64748B),
          fontSize: 13,
          fontWeight: FontWeight.bold,
          letterSpacing: 2,
        ),
      ),
```

```
      const SizedBox(height: 8),

      Text(
        result,
        style: const TextStyle(
          color: Colors.white,
          fontSize: 42,
          fontWeight: FontWeight.bold,
        ),
      ),
    ],
  ),
),

const SizedBox(height: 25),

// --------------------------------
// FIRST NUMBER
// --------------------------------

numberField(
  'First Number',
  'Enter first number',
  firstController,
  Icons.looks_one_rounded,
),

const SizedBox(height: 16),

// --------------------------------
// SECOND NUMBER
// --------------------------------
```

```
numberField(
  'Second Number',
  'Enter second number',
  secondController,
  Icons.looks_two_rounded,
),

const SizedBox(height: 25),

// --------------------------------
// ADDITION & SUBTRACTION
// --------------------------------

Row(
  children: [

    operationButton(
      'Addition',
      Icons.add,
      () => calculate('+'),
    ),

    operationButton(
      'Subtraction',
      Icons.remove,
      () => calculate('-'),
    ),
  ],
),

// --------------------------------
```

```
// MULTIPLICATION & DIVISION
// --------------------------------

Row(
  children: [

    operationButton(
      'Multiplication',
      Icons.close,
      () => calculate('×'),
    ),

    operationButton(
      'Division',
      Icons.call_split,
      () => calculate('÷'),
    ),
  ],
),

const SizedBox(height: 12),

// --------------------------------
// CLEAR BUTTON
// --------------------------------

SizedBox(
  width: double.infinity,
  height: 60,

  child: ElevatedButton.icon(
    onPressed: clearCalculator,
```

```
      icon: const Icon(
        Icons.refresh_rounded,
      ),

      label: const Text(
        'CLEAR',
        style: TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.bold,
          letterSpacing: 1,
        ),
      ),

      style: ElevatedButton.styleFrom(
        backgroundColor:
            const Color(0xFFEF4444),

        foregroundColor: Colors.white,

        elevation: 0,

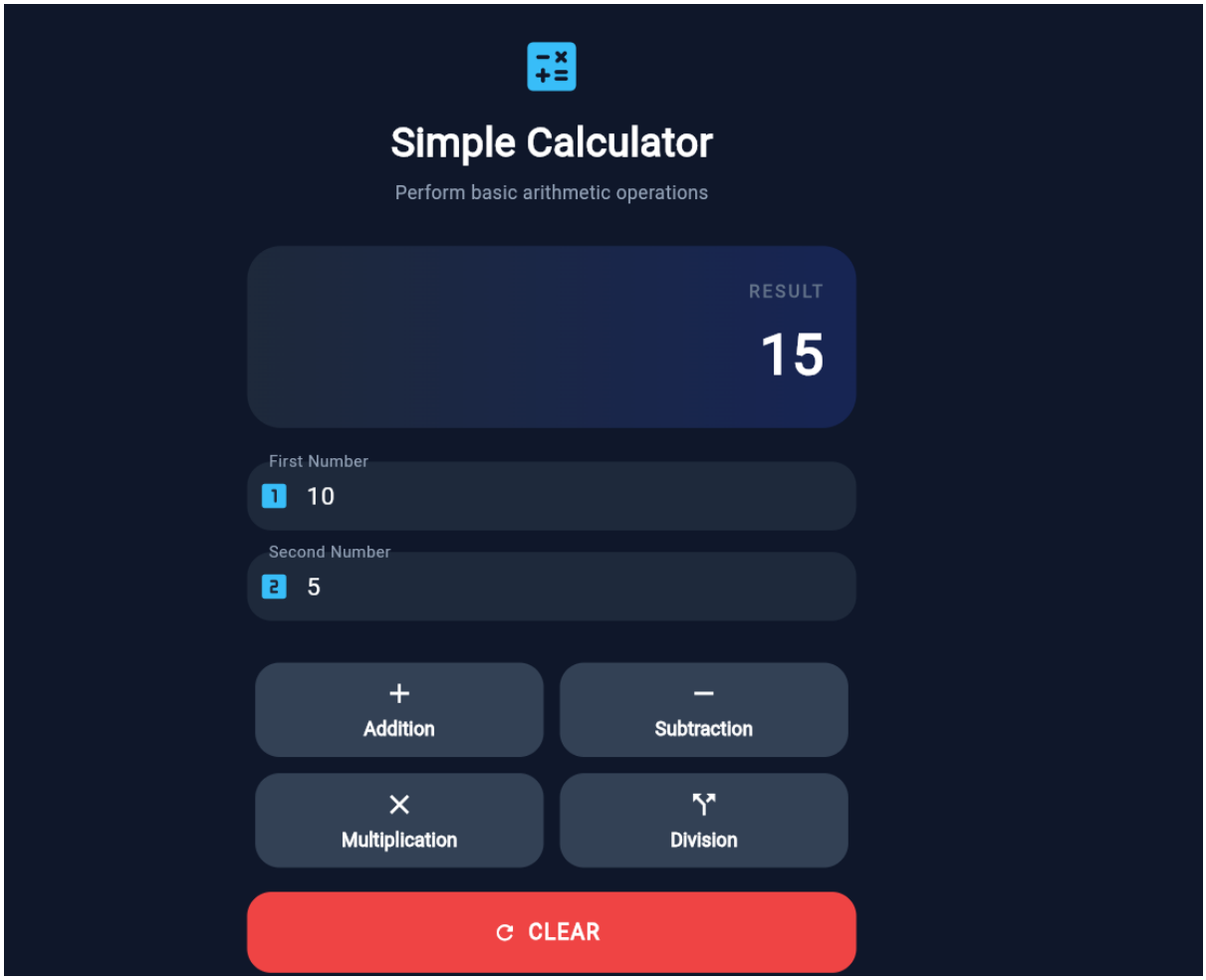
        shape: RoundedRectangleBorder(
          borderRadius:
              BorderRadius.circular(18),
        ),
      ),
    ),
  ),

  const SizedBox(height: 25),
```

```
                    // --------------------------------
                    // FOOTER
                    // --------------------------------

                    const Text(
                      'Flutter • Dart',
                      style: TextStyle(
                        color: Color(0xFF475569),
                        fontSize: 13,
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

OUTPUT



RESULT

Thus, the Flutter application to implement a Simple Calculator was developed and executed successfully. The application accepts two numbers, performs Addition, Subtraction, Multiplication, and Division, displays the result, and provides a Clear button to reset the input fields and result.